DualNode: seção do Host

Responsável: [nome do integrante A] · Módulo: peer/host.py · Rascunho para o relatório técnico. Revise com as palavras do grupo e troque os números pelos logs das execuções finais.

1. Papel do host e decisão de arquitetura

Em cada partida, o jogador que cria a sala (slot 0) atua como **host**, o árbitro da partida. Ele é um par como os outros: roda no processo de um jogador e joga com o seu próprio teclado. O que o distingue é ser a **única autoridade sobre o estado do jogo**: só o host executa as regras de cada ciclo e publica o resultado; os demais jogadores enviam a direção desejada e aplicam, em ordem, o que o host publica.

Justificativa. O enunciado exige que todos cheguem ao mesmo resultado (quais motos colidiram, em que ciclo, quem venceu). Com uma única autoridade, a consistência vale por construção. A alternativa seria uma simulação distribuída (*lockstep*), em que todos calculam o estado:

	Host autoritativo (adotado)	Lockstep
Consistência	Garantida: um único cálculo	Exige as entradas de todos em cada ciclo
Atraso de um jogador	Não trava os demais	O ciclo só avança com a entrada do mais lento
Queda de jogador	O host decide e publica um único DISC	Todos precisam concordar em qual ciclo ele saiu
Complexidade	Baixa	Alta
Ponto único de falha	Sim (o host)	Não

O ponto único de falha é aceito pelo enunciado, desde que a falha do host encerre a partida de forma controlada (seção 8). O custo é a latência: a direção de um jogador leva meia ida e volta até o host, e o resultado leva outra meia de volta. Em rede local isso é bem menor que um ciclo (100 ms). O host joga com latência zero, uma vantagem conhecida (seção 12).

A topologia é em estrela: durante a partida todo o tráfego passa pelo host, e o servidor de lobby não participa.

2. Parâmetros

Parâmetro	Valor padrão	Origem
tick_ms	100 ms (10 ciclos por segundo)	Enviado pelo lobby no MATCH_START
Contagem regressiva	3000 ms (30 TICKs vazios)	Constante COUNTDOWN_MS
hello_deadline_ms	10000 ms	Configuração (timeouts)
peer_timeout_ms	3000 ms (30 INPUTs seguidos perdidos)	Configuração (timeouts)
JANELA	8 registros de ciclo por TICK	Constante
MAX_DATAGRAMA	1200 bytes	Constante
TOLERANCIA_ATRASO	5 ciclos	Constante
ABORT	3 envios, 50 ms entre eles	Método _abortar

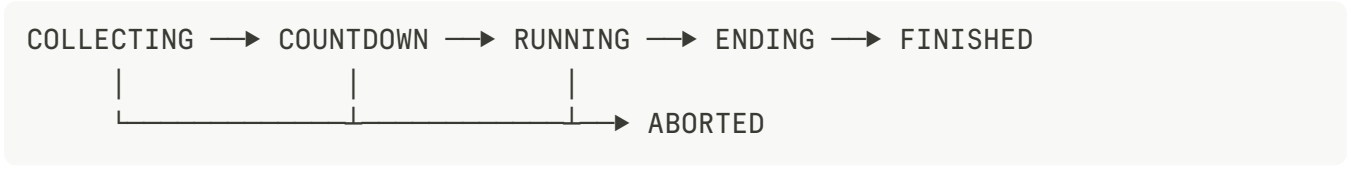
3. Mensagens do host (UDP)

Cada datagrama é um objeto JSON em UTF-8 com os campos comuns `v` (versão, 1), `t` (tipo), `m` (identificador da partida) e `pid` (slot do remetente). O limite é de 1200 bytes, para evitar fragmentação IP.

Mensagem	Sentido	Campos	Significado
HELLO	peer → host	pid	Estou pronto. Reenviado pelo peer até chegar o WELCOME
WELCOME	host → peer	state, countdown_ms	Estado inicial da arena e duração da contagem. O host responde a todo HELLO, inclusive repetido
INPUT	peer → host	iseq, dir, ack	Direção desejada (U, D, L ou R), número de sequência e último ciclo aplicado. Enviado a cada ciclo, mesmo sem mudança
TICK	host → peer	recs	Registros de ciclo. Também serve de sinal de vida, mesmo com recs vazio
ABORT	host → peers	reason	O host encerrou a partida de forma controlada (melhor esforço)

Um **registro de ciclo** tem a forma `{n, mv, el, end}`: `n` é o número do ciclo (consecutivo, a partir de 1); `mv` lista as novas posições `[pid, x, y]` de quem se moveu; `el` lista as eliminações `[pid, motivo]` com motivo WALL, TRAIL, HEAD ou DISC; `end` é nulo ou o resultado da partida (vitória de um jogador ou empate).

4. Máquina de estados



Estado	O que o host faz	Sai quando
COLLECTING	Responde HELLO com WELCOME e manda TICK vazio, a cada tick_ms, aos peers que já se apresentaram	Todos se apresentaram, ou o prazo hello_deadline acabou
COUNTDOWN	Manda 30 TICKs vazios, um por tick_ms	A contagem termina
RUNNING	A cada tick_ms: lê os INPUT e o teclado, executa o ciclo, guarda o registro e envia os TICKs	Um registro traz o campo end

Estado	O que o host faz	Sai quando
ENDING	Reenvia o registro final apenas a quem ainda não confirmou	Todos confirmaram (ack igual ao último ciclo) ou o peer_timeout acabou
FINISHED	Fecha o socket e devolve o resultado	
ABORTED	Avisa os peers com ABORT e encerra	

O ABORTED é alcançado por saída do host (tecla Q ou Ctrl+C) antes do fim da partida, ou quando nenhum peer se apresenta.

5. Mensagens inválidas, inesperadas ou ausentes

Todo datagrama passa por uma sequência de checagens, das mais baratas para as mais caras. Se alguma falhar, ele é **descartado em silêncio** e o motivo vai para o log de depuração. O host nunca responde com erro a um datagrama UDP inválido: isso evitaria amplificação e conversas em laço entre implementações.

#	Checagem	Descarta quando
1	Tamanho	Maior que 1200 bytes
2	Decodificação	Não é UTF-8, não é JSON, não é objeto, versão diferente de 1 ou falta campo
3	Partida	O campo m é de outra partida
4	Identificador	O pid não é um número inteiro
5	Origem	O endereço (IP e porta) não pertence a nenhum peer da partida
6	Coerência	O pid informado não é o do endereço de origem (um peer se passando por outro)
7	Estado	O tipo de mensagem não é aceito no estado atual (por exemplo HELLO durante o jogo)

Dentro de um INPUT válido ainda se exige: `iseq` e `ack` inteiros (booleanos não valem) e `ack` não negativo; `dir` em U, D, L ou R; e `iseq` maior que o último aceito. Um INPUT repetido ou atrasado é descartado. O `ack` aceito nunca passa do tamanho do histórico e nunca retrocede.

Sobre mensagens **ausentes**, o host usa prazos em todos os pontos de espera (hello_deadline, peer_timeout e o tempo do ENDING), de modo que nenhum jogador fica travado indefinidamente.

6. O ciclo

A cada tick_ms, o host executa nesta ordem: (1) verifica os timeouts de peers; (2) monta as direções desejadas (a última válida de cada peer e a do teclado do host); (3) chama o cálculo das regras, que devolve o registro do ciclo; (4) guarda o registro no histórico; (5) envia os TICKs; (6) atualiza e desenha a própria tela. Os TICKs saem **antes** do desenho: a rede tem prioridade sobre a tela.

O relógio do ciclo usa `time.monotonic()` (que não salta com ajustes do relógio do sistema) e agenda o próximo ciclo somando o intervalo ao instante anterior, e não ao instante atual. Assim, um atraso pontual não se acumula. Se o host ficar atrasado mais de 5 ciclos, o relógio é reajustado em vez de rodar vários ciclos seguidos. Entre ciclos, o laço dorme no máximo 5 ms, para continuar lendo a rede e o teclado.

O host trabalha em uma única thread, sem bloqueio: o socket é não bloqueante e, a cada volta, esvazia todos os datagramas pendentes antes de decidir se é hora de um ciclo.

7. Confiabilidade sobre UDP

O UDP não garante entrega, ordem nem ausência de duplicatas. O protocolo implementa o necessário na aplicação, **sem retransmissão por temporizador**, aproveitando que as mensagens descrevem estado:

- **Ordem e duplicatas.** Os registros têm número de ciclo consecutivo. O peer aplica apenas o registro seguinte ao último aplicado; duplicados são descartados e registros do futuro esperam o anterior.
- **Recuperação de perdas.** Todo INPUT carrega o ack (último ciclo aplicado). A cada ciclo, o host envia a cada peer os registros de ack+1 até ack+8, cortando o fim da lista se o datagrama passar de 1200 bytes. Se um TICK se perde, o seguinte já repete o que faltou, e um peer atrasado se atualiza até 8 ciclos por datagrama. Cada peer recebe uma janela diferente, conforme o seu ack.
- **Entrada idempotente.** O INPUT diz qual é a direção desejada (estado), não que houve uma mudança (evento). A perda de um INPUT é compensada pelo seguinte, e o host usa o de maior iseq.
- **Handshake.** O peer repete o HELLO até receber o WELCOME, e o host responde a todo HELLO. Perder qualquer um dos dois se resolve com a repetição.

- **Sinal de vida.** O host manda TICK, mesmo vazio, desde o WELCOME até o fim; o peer manda INPUT a cada ciclo. Cada lado detecta a queda do outro pelo silêncio.
- **Fim confiável.** O último TICK, o que carrega o resultado, também pode se perder. Por isso o host entra em ENDING e continua enviando o registro final, só a quem ainda não confirmou, até o ack chegar ao último ciclo ou o prazo acabar. O peer, por sua vez, continua mandando INPUT por cerca de 2 s depois de ver o fim.
- **Encerramento.** O ABORT é enviado 3 vezes, espaçado em 50 ms, apenas a peers conectados. É melhor esforço: se todos se perderem, o peer detecta a falta de TICK pelo seu próprio timeout.

8. Tolerância a falhas

Falha	Detecção	Comportamento definido
Peer cai durante a partida	Sem INPUT válido por peer_timeout (3 s)	A moto é eliminada com DISC no ciclo seguinte; o rastro permanece e a partida continua. Ocorre uma vez só; depois o host para de enviar TICK a ele e ignora seus datagramas
Peer cai e só resta o host	O mesmo	O host vence
Peer não se apresenta	Sem HELLO até hello_deadline	A moto é eliminada com DISC no ciclo 1. Se nenhum peer se apresentar, o host aborta com o motivo jogadores insuficientes
Peer entra devagar	Os outros já se apresentaram	O host manda TICK vazio aos que já entraram, para que não desistam durante a espera
Peer já eliminado no jogo e depois some	Sem INPUT	Nenhum DISC falso: só se elimina quem ainda está vivo
Perda de pacotes	ack atrasado no host	Recuperação pela janela de registros
Último TICK perdido	ack inferior ao último ciclo	O host reenvia até confirmar (ENDING)
Host sai (tecla Q ou Ctrl+C)	Ação do usuário	ABORT aos peers em qualquer fase antes do fim. Depois do fim da partida, o resultado real é preservado

Falha	Detecção	Comportamento definido
Host cai sem avisar	Peers: sem TICK por host_timeout (3 s)	Cada peer encerra de forma controlada. Não há eleição de novo host
Lobby cai	Não afeta o host	A partida segue, porque o jogo não usa o TCP do lobby

9. Consistência de estado

O host calcula cada ciclo uma única vez e publica um registro com número consecutivo. Cada peer aplica a mesma sequência de registros, na mesma ordem, e o resultado final vem no campo end do último deles. Duas propriedades garantem que nenhum peer diverge: a ordem (só se aplica o registro n+1) e a recuperação (o host reenvia até o peer confirmar). Em todos os cenários com rede (seção 10), comparamos o resultado impresso pelo host com o resumo de cada peer, inclusive com perda de pacotes e reordenação, e eles foram idênticos.

10. Testes e resultados

Os números abaixo vêm de execuções em localhost e com IPs distintos de loopback. As regras do jogo usadas foram as provisórias (só colisão com a parede), então os ciclos específicos mudam quando a lógica completa entrar; o critério que vale é o resultado **idêntico em todos os processos**.

10.1 Testes unitários (sem rede)

Tema	O que verifica
Tabela de jogadores e origem	Peers por pid, host fora da tabela, origem validada por IP e porta
Validação de datagramas	12 verificações: lixo, versão, partida, passar-se por outro, origem, tipo, tamanho, estado
Timeout de peer	8 verificações: DISC único, rastro mantido, TICK deixa de ser enviado, INPUT tardio ignorado, sem DISC falso
Fim confiável	6 cenários: janela correta para peer atrasado, confirmado deixa de receber, peer mudo não trava o host

Tema	O que verifica
Teclado e tela	11 verificações: direção do host, inversão ignorada, tela e contagem
Saídas do host	11 verificações: tecla Q e Ctrl+C em cada fase, ABORT enviado uma vez só, porta ocupada
Rede simulada	10 verificações: perda, atraso, jitter com reordenação, semente reproduzível
Sinal de vida na espera	5 verificações

10.2 Cenários com rede

Cenário	Resultado
Contagem regressiva	30 TICKs com intervalo médio de 100,0 ms (mínimo 99,8 e máximo 100,3), sem deriva
Partida completa, 3 processos	Host e peers com o mesmo resultado e as mesmas eliminações
30% de perda de TICKs nos peers	17 e 23 TICKs descartados de propósito; mesmo resultado nos 3 processos
Direção do peer por INPUT	A moto muda de direção e a inversão de sentido é ignorada pelo host
Queda de um peer	DISC no ciclo 9 (com timeout de 800 ms nos testes; o log registra silêncio de 0,9 s); a partida continua para os outros
Queda dos dois peers	Os dois DISC no mesmo ciclo; o host vence
Último TICK perdido (antes do ENDING)	O peer termina sem saber o resultado (fim nulo)
Último TICK perdido (com ENDING)	Peer descarta 3 TICKs do fim: confirmação em cerca de 300 ms e resultado correto. Se descarta todos, o host desiste em 0,8 s (o prazo) e devolve o resultado
Saída do host (Q na contagem, Q no jogo, Ctrl+C no jogo)	Os peers recebem o ABORT imediatamente com o motivo; o host sai com código 0
Saída do host esperando jogadores	Sai na hora, sem erro

10.3 Rede ruim simulada

O simulador (perda, atraso e jitter, nos dois sentidos) fica no socket do host, e como todo o tráfego passa por ele, cobre todos os caminhos.

Cenário	Resultado
20% de perda	Mesmo resultado nos 3 processos. O simulador descartou 31 de 146 datagramas recebidos e 31 de 133 enviados; um HELLO precisou ser repetido; o fim foi confirmado em 5 ms
20% de perda, 80 ms de atraso e jitter de 40 ms	Mesmo resultado; fim confirmado em cerca de 170 ms
50% de perda	Mesmo resultado, com 88 de 150 datagramas recebidos perdidos e nenhum DISC falso (timeout de 3 s)
Atraso de 600 ms em cada sentido	Peers com o resultado correto, mas atrasados; o host desistiu de esperar a confirmação depois de 3 s
Atraso de 1000 ms em cada sentido	Os peers **não** souberam o resultado (limite do protocolo, seção 12)

10.4 IPs distintos

Com o arquivo de MATCH_START gerado com IPs diferentes (127.0.0.1, 127.0.0.2 e 127.0.0.3, que o host trata como máquinas distintas porque valida a origem pelo IP):

Cenário	Resultado
Partida normal	Mesmo resultado nos 3 processos
IP errado de um peer no MATCH_START	O log do host mostra origem desconhecida; o peer é eliminado com DISC no ciclo 1; a partida continua para os outros e termina com o resultado correto
Com 20% de perda e 80 ms de atraso	Mesmo resultado nos 3

A preencher pelo grupo: repetir o cenário de partida completa com pelo menos duas máquinas físicas na mesma rede e anexar os logs.

11. Dificuldades encontradas

- **Deriva do relógio.** Dormir um intervalo fixo por ciclo acumula atraso. A solução foi usar o relógio monotônico e agendar cada ciclo a partir do agendamento anterior.
- **O último datagrama perdido.** Fechar o socket assim que o ciclo final acontece deixava peers sem o resultado. Isso levou à fase ENDING.
- **Peer lento desistindo durante a espera.** Só apareceu ao testar com IPs diferentes: um peer que se apresentava cedo ficava 8 s sem receber nada e desistia antes de a partida começar. Corrigido com o TICK vazio durante a espera.
- **Origem validada por IP e porta.** Cada peer precisa enviar do mesmo socket em que escuta, com bind na porta anunciada; um socket de envio novo teria porta de origem aleatória e seria descartado.
- **Interface e log.** Com a tela de terminal aberta, `print` estraga o desenho; todo o diagnóstico vai para arquivo de log.
- **Janela contra atraso da rede.** Os testes de atraso mostraram o limite descrito na seção 12.

12. Limitações conhecidas

- **Ponto único de falha.** Se o host cai, a partida termina. Não há eleição de novo host.
- **Latência do host.** O host joga com latência zero, e os peers com pelo menos uma ida e volta.
- **Janela contra atraso.** O ack que o host conhece está sempre uma ida e volta atrasado, e a janela cobre 8 ciclos (800 ms). Com atraso de ida e volta menor que isso, tudo funciona. Com 1,2 s os peers ficam atrasados; com 2 s não alcançam o host antes de ele encerrar. Em rede local o atraso é de poucos milissegundos, e a saída seria aumentar a janela (cabem cerca de 14 registros em 1200 bytes) ou compactar o formato.
- **Fim sem garantia absoluta.** O host nunca tem certeza de que um peer soube o resultado: se todos os reenvios se perderem, ele desiste depois do prazo (é o problema dos dois generais). A chance disso é pequena: com 30% de perda, perder 8 TICKs seguidos é cerca de 0,007%.
- **Sem autenticação.** A validação por IP e porta protege contra interferência acidental, não contra um participante malicioso.
- **Sem controle de congestionamento.** Aceitável nesta escala (poucos jogadores, datagramas pequenos).
- **Host de uma thread.** Durante a partida o processo não atende o TCP do lobby; isso exige o alinhamento descrito na seção 13.

13. Relação com o restante do sistema

O host lê da configuração as chaves `peer.bind_host`, `timeouts.hello_deadline_ms`, `timeouts.peer_timeout_ms` e, só para teste, `sim`. Recebe a mensagem MATCH_START do lobby, cujo campo `config` é a arena (largura, altura e tick_ms). Os endereços dos jogadores no MATCH_START precisam ser **IPs numéricos alcançáveis**: nomes como localhost, ou 0.0.0.0, quebram a validação de origem.

Como o host não lê o TCP durante a partida, o lobby não pode derrubar jogadores que estão em partida por falta de resposta ao PING. A decisão adotada é que o cliente fecha o TCP logo depois do MATCH_START e o lobby trata esse fechamento como normal para quem está em jogo. **[Confirmar com a pessoa responsável pelo lobby.]**

O resultado devolvido pelo host ao programa principal é um dicionário com `result` (vitória, empate ou ABORT) e `winner` (o pid do vencedor ou nulo).